DA2304 Assignment 2

VM Architecture and Matrix Operations (MAC)

Name: Venkata Sai Teja Madineni Roll Number: DA24B031

April 30, 2026

1 Part 2: Matrix Operations (MAC)

1.1 Sanity Checking

Before performing matrix multiplication, we need to ensure that the operation is valid. This is done by checking that the number of columns of W is equal to the number of rows of X (i.e., $W_{\text{cols}} = X_{\text{rows}}$).

After computing $W \cdot X$, we add the bias matrix B. For this addition to be valid, both matrices must have the same dimensions. Since the result of $W \cdot X$ has dimensions $(W_{\text{rows}} \times X_{\text{cols}})$, we must ensure that $B_{\text{rows}} = W_{\text{rows}}$ and $B_{\text{cols}} = X_{\text{cols}}$.

Instead of using high-level exception handling, we follow a low-level approach consistent with Hack architecture. We use a dedicated memory location (R15) as an error flag.

```
ERROR_FLAG_ADDR = 15

func sanity_check(W, X, B):

‘‘‘
MEMORY[ERROR_FLAG_ADDR] = 1

W_rows = rows(W)
W_cols = cols(W)

X_rows = rows(X)
X_cols = cols(X)

B_rows = rows(B)
B_cols = cols(B)

if W_cols != X_rows:
    MEMORY[ERROR_FLAG_ADDR] = -1
```

```
if B_rows != W_rows:
    MEMORY[ERROR_FLAG_ADDR] = -1

if B_cols != X_cols:
    MEMORY[ERROR_FLAG_ADDR] = -1

return MEMORY[ERROR_FLAG_ADDR]
```

A value of 1 indicates valid dimensions, while -1 indicates an error.

1.2 Approach A: MATMUL Function

In this approach, the entire matrix multiplication is done inside a single function. First, we allocate memory for the output matrix M of size $(W_{\text{rows}} \times X_{\text{cols}})$. Then we use three nested loops.

The outer two loops iterate over each position (i, j) of the output matrix. For each position, we compute the dot product of the i-th row of W and the j-th column of X. This is done using the inner loop, where we accumulate the value:

$$\text{val} = \sum_{k} W[i][k] \cdot X[k][j]$$

This value is stored in $M[i][j]$. After computing the full matrix $M = W \cdot X$, we call a separate function to add the bias matrix B element-wise.

```
func MATMUL(W, X):

W_rows = rows(W)
W_cols = cols(W)
X_cols = cols(X)
M = allocate_memory(W_rows, X_cols)
for i from 0 to W_rows - 1:
    for j from 0 to X_cols - 1:
        val = 0
        for k from 0 to W_cols - 1:
            val = val + (W[i][k] * X[k][j])
        M[i][j] = val
return M
function ADD_BIAS(M, B):
    rows = rows(M)
    cols = cols(M)
    Y = allocate_memory(rows, cols)
    for i from 0 to rows - 1:
        for j from 0 to cols - 1:
            Y[i][j] = M[i][j] + B[i][j]
    return Y
```

Execution flow for Approach A:

```
validation = sanity_check(W, X, B)

if validation == -1:
exit

M = MATMUL(W, X)
Y = ADD_BIAS(M, B)
```

1.3 Approach B: ROWXCOL Function

In this approach, instead of computing the entire multiplication inside one function, we compute each element separately using a helper function (`ROWXCOL`).

First, we allocate memory for the output matrix Y of size $(W_{\text{rows}} \times X_{\text{cols}})$.

We iterate using two loops over i and j. For each position (i, j), we call the function `ROWXCOL`, which computes the dot product of the i-th row of W and the j-th column of X.

Inside `ROWXCOL`, we use a loop to compute:

$$\text{num} = \sum_k W[i][k] \cdot X[k][j]$$

The computed value is returned to the main function, and then we add the bias $B[i][j]$ and store the result in $Y[i][j]$.

```
func ROWXCOL(row_ptr, col_ptr, length, stride):

    num = 0

    for k from 0 to length - 1:
        num = num + (row_ptr[k] * col_ptr[k * stride])

    return num
```

Execution flow for Approach B:

```
validation = sanity_check(W, X, B)

if validation == -1:
    exit

W_rows = rows(W)
W_cols = cols(W)
X_cols = cols(X)

Y = allocate_memory(W_rows, X_cols)

for i from 0 to W_rows - 1:
    for j from 0 to X_cols - 1:
        value = ROWXCOL(W[i], base(X) + j, W_cols, X_cols)
        Y[i][j] = value + B[i][j]
```

2 Architectural Analysis

2.1 Stack Usage

In Hack VM, the stack is used to store function arguments, return addresses, and local variables.

Approach A (MATMUL): In this approach, the MATMUL function is called only once from the main program. The required inputs (pointers to W, X, B and dimensions) are pushed onto the stack only once.

Inside the function, the computation is done using nested loops with VM commands like `label`, `goto`, and `if-goto`. There are no additional function calls. The stack is mainly used for loop counters, accumulators, and temporary values.

Since only one stack frame is created and reused throughout the computation, the number of push/pop operations is small, and the overhead is low.

Approach B (ROWXCOL): In this approach, the ROWXCOL function is called for every output element. So for an $m \times n$ matrix, the function is called $m \times n$ times.

For each function call:

- Arguments (row pointer, column pointer, and length) are pushed onto the stack
- The `call` instruction pushes the return address and saves the current frame (LCL, ARG, THIS, THAT)
- A new stack frame is created for the function
- After computation, the frame is restored using `return`

This repeated creation and destruction of stack frames leads to a large number of push and pop operations. The stack is also used inside the function for the dot-product computation.

So Approach A has much lower stack usage and function call overhead. Approach B introduces high overhead due to repeated function calls and stack frame management.

2.2 Segment Usage

Hack VM has segments like argument, local, temp, pointer, and static.

Approach A (MATMUL): In this approach, the argument segment is set up only once when the function is called. Local variables (like loop counters and accumulators) are reused throughout the computation.

Pointer segments are updated efficiently to traverse rows and columns of the matrices. Since there are no repeated function calls, there is minimal overhead of saving and restoring segment pointers (LCL, ARG, THIS, THAT).

Approach B (ROWXCOL): In this approach, for every function call, the argument segment is set up again with new row and column pointers.

The outer computation maintains its own local variables (like i, j),

while the ROWXCOL function uses its own argument and local segments (for pointers and the loop variable k).

This results in frequent saving and restoring of segment pointers (LCL, ARG, THIS, THAT). Pointer segments are also updated repeatedly during each dot-product computation.

So Approach A uses memory segments more efficiently. Approach B increases segment usage and memory access due to repeated setup and restoration of segment pointers.

2.3 Optimization (Approach A): MATMUL

Approach A already benefits from low function call overhead, but the inner triple-nested loop (especially the dot-product) dominates execution time on the Hack architecture.

- Instead of computing the full $W \cdot X$ matrix and then calling a separate ADD_BIAS function, the bias addition can be integrated into the outer loops. After computing each dot product for (i, j), we directly add $B[i][j]$. This removes an additional full pass over the matrix and reduces memory access.
- Pre-compute base addresses and use pointer variables that increment during iteration instead of recalculating absolute offsets like $W[i][k]$ and $X[k][j]$ in every step. This reduces repeated arithmetic operations.
- For small or bounded values of k, the inner loop can be partially unrolled to reduce loop control overhead (label, goto, if-goto), improving execution speed.
- Store matrices in a layout that simplifies traversal (for example, keeping rows contiguous), reducing pointer updates and address recomputation during iteration.
- Since the Hack ALU does not support native multiplication, the operation $W[i][k] \cdot X[k][j]$ must be implemented using repeated addition or bitwise methods. Using an optimized multiplication routine significantly improves performance.
- The VM translator can generate tighter assembly by reducing unnecessary push/pop operations and keeping frequently used values (such as accumulators and indices) in temp segments or near the top of the stack.

2.4 Optimization (Approach B): ROWXCOL

Approach B suffers from high overhead because ROWXCOL is invoked for every element of the output matrix. Each invocation requires setting up arguments, saving and restoring segment pointers (LCL, ARG, THIS, THAT), and performing call/return jumps.

- Function inlining is the most effective optimization. Instead of generating a call to ROWXCOL, the compiler can directly insert the dot-product computation into the main loop. This removes the overhead of repeated function calls and stack frame management, making execution closer to Approach A.
- Values that do not change within inner loops (such as base addresses of a row or column) can be computed once outside the loop and reused.
- The VM translator can remove redundant instruction patterns (for example, a push immediately followed by a pop) in the critical execution path.
- Expensive computations used in indexing can be replaced with simpler operations like pointer increments.
- Frequently accessed values can be kept in temp segments or reused on the stack to reduce repeated accesses to the argument segment.

Extra Credit

Recent Favorite Song: yetta yetta song from lenin movie